



1 · SETUP

installer la chaîne, une fois pour toutes

npm i -g @anthropic-ai/claude-code

installe Claude Code

claude --version

vérifie, puis claude dans le dépôt

claude mcp add playwright

branche Playwright

npm x @playwright/mcp@latest

MCP

claude mcp list

doit afficher playwright

npm x playwright install --with-deps

installe les navigateurs

npm i -D @playwright/test @axe-core/playwright

E2E + accessibilité

dotnet add package xunit FluentAssertions NSubstitute

socle de test .NET

dotnet tool install -g dotnet-stryker

mutation testing

dotnet tool install -g dotnet-coverage

couverture .NET

docker compose up -d && dotnet test

base éphémère, puis la suite

2 · CLAUDE CODE — L'ESSENTIEL

ce qu'on tape vraiment quand on teste

claude

ouvre une session dans le dépôt

/status

compte authentifié + répertoire

/context

usage du contexte, gains suggérés

/clear

après 2 corrections identiques

/compact [consignes]

résumé, libère du contexte

--permission-mode plan

la stratégie avant tout code

--permission-mode dontAsk

mode CI verrouillée

Ctrl+B

passse dotnet test en arrière-plan

Esc Esc

rewind (Alt+M si VT inactif)

/agents

ne crée plus : éditer .claude/agents/*.md

/verify

build, lance l'app et observe

/code-review [--fix]

revue multi-agents du dépôt

/security-review

revue sécurité, sur demande explicite

claude -p "" --output-format json

headless ; rend total_cost_usd

--bare

ignore hooks, MCP, CLAUDE.md → CI

--max-turns N

aucune limite par défaut : plafonner

CLAUDE.md racine < 200 lignes

seul fichier qui survit à la compaction

permissions.deny

.claudeignore n'existe pas

Pièges outils. Write(path) et Glob(path) sont acceptées mais jamais appliquées — seules Edit(path) et Read(path) le sont. Checkpoints : 100/session, purge à 30 j, les écritures par bash ne sont pas tracées. Git reste le filet.

3 · MCP POUR LA QA

donner des yeux à l'agent

claude mcp add playwright ...

ajoute le serveur

claude mcp list · /mcp

liste et état des serveurs

--transport http

sse est déprécié

.mcp.json

versionné, relu en PR comme du code

Arbre d'accessibilité

pas de pixels, pas de modèle vision

Snapshot ≈ 200-400 tokens

référence stable par élément (e5)

69 outils browser_*

generate_locator, route...

Chrome DevTools MCP

45 outils : LCP/INP/CLS, réseau, Lighthouse

GitHub MCP

get_job_logs failed_only:true

Retirer après génération

69 schémas coûtent du contexte

La description d'un outil est du prompt injecté, pas de la documentation.

4 · ANATOMIE D'UN PROMPT DE TEST

cinq blocs — et <task> toujours en dernier

<role>

fixe le référentiel : ISTQB, xUnit, WCAG

<documents>

vérité hors du code — toujours en haut

<constraints>

interdictions vérifiables, pas des vœux

<output_format>

chemin exact, sortie parsable

<examples>

1 = un style · 3 à 5 = une taxonomie

<role>Ingénieur QA senior. ISTQB CTFL v4.0.1, xUnit v3 + FluentAssertions.</role>

<documents><document index="1"><source>{{CHEMIN_SPEC}} - {{SECTION}}</source>

<document_content>{{CONTENU_SPEC}}</document_content></documents>

<constraints>

- N'ouvre PAS {{FICHIER_SOUS_TEST}}. La vérité est le document 1.

- Un comportement = un test. Interdit de mocker {{INTERFACE_DOMAINE}}.

- Chaque [Fact] précédé de // {{REF_SPEC}} : « citation littérale ».

- Ambiguïté → [Fact(Skip = "ambiguïté {{ID}}")] + question au métier.

</constraints>

<output_format>Un seul fichier, chemin {{CHEMIN_FICHIER_TEST}}.</output_format>

<task>Produis la suite de {{CLASSE}} à partir du document 1 seul.</task>

Documents > 20 k tokens en haut, tâche en bas : jusqu'à +30 % de qualité.

5 · PROMPTS PRÊTS À L'EMPLOI

annexe B, condensée — un prompt par ligne

P-04 unit .NET

« Produis la suite de {{CLASSE}} depuis le doc 1 seul. N'ouvre pas le code. »

P-08 E2E

« Écris l'E2E de {{PARCOURS}} : locators lus dans le snapshot, zéro waitForTimeout. »

P-12 Gherkin

« Génère les scénarios des exigences du doc 1, un par comportement, traçables. »

P-14 données

« Générateur de {{ENTITE}} : graine constante, domaines .test, partitions couvertes. »

P-17 échec

« Constitue le dossier de {{TEST}} en 7 sections, sans hypothèse. »

P-19 flakiness

« Classe les échecs du doc 1 dans la taxonomie, avec discriminant et n/N. »

P-21 revue

« Révise la suite : 8 points, verdict OK / À CORRIGER / BLOQUANT + preuve citée. »

P-23 charge k6

« Scénario de {{ENDPOINT}} : arrival-rate seul, seuils p(95)/p(99), exit non nul. »

P-26 a11y

« Audit de {{COMPOSANT}} : test axe + clavier seul + ce qu'ils ne voit pas. »

P-27 synthèse

« Rédige la synthèse de {{CAMPAGNE}} depuis les sorties structurées, chiffres sourcés. »

4 règles non négociables : toute valeur attendue cite sa source · interdiction de lire le code testé · au moins un {{PLACEHOLDER}} · aucune sortie n'est réputée exécutée avant lecture du runner.

6 · COMMANDES DE TEST

.NET / xUnit · Angular / Playwright · k6

dotnet test

la suite complète

--filter

cible une

"FullyQualifiedName-VatCalculator"

classe

--minimum-expected-tests 47

code 9 si des tests ont disparu

--coverage --coverage-output-format cobertura

couverture exploitable

dotnet stryker --threshold-break 40

score de mutation

dotnet-counters collect --format csv

GC, thread pool côté serveur

ng test --no-watch --no-progress

front en CI (Vitest + jsdom)

npm playwright test

la suite E2E

npm playwright test --ui

mise au point interactive

--repeat-each=20 --workers=4

chasse au flaky

--trace=on

--workers=1

discriminant « concurrence »

--shard=1/4 --reporter=blob

parallélisation en CI

npm playwright merge-reports ./all-blob-reports

fusion des shards

k6 run perf/search-breakpoint.ts

sortie non nulle si seuil dépassé

TZ=Pacific/Auckland dotnet test — le discriminant le plus rentable. --shard n'équilibre que si fullyParallel: true ; sinon le découpage est au fichier près.

7 · LES 6 ANTI-PATTERNS

symptôme observable → test de détection

Test tautologique	100 % vert du 1 ^{er} coup sur du code jamais testé	l'attendu est-il citable dans la spec ?
Sélecteur halluciné	locator plausible, absent du DOM réel	grep du locator dans le snapshot
Couverture trompeuse	couverture en hausse, zéro assertion nouvelle	publier couverture et score de mutation
Sur-mock	le test casse au refactoring, pas au bug	supprimer le mock : le test tient-il ?
Paquet halluciné	add package d'un nom inconnu	vérifier sur nuget.org / npmjs.com
Retry qui masque	retries: 3, pipeline vert, moral en hausse	--repeat-each=20 sur le même code

8 · CHECKLIST AVANT COMMIT

10 cases opposables — le Copilote les fait cocher

- ☐ Chaque test **cite sa source** (`// CDC v4.0 $x.y : « ... »`), littéralement.
- ☐ Aucune valeur attendue n'est justifiable uniquement par le code.
- ☐ Au moins un test a été **rouge** avant d'être vert.
- ☐ La sortie du runner a été lue : générés / compilent / passent.
- ☐ Un comportement = un test ; aucun test à cinq assertions.
- ☐ Zéro `waitForTimeout`, zéro sélecteur positionnel (grep à l'appui).
- ☐ Aucun test supprimé ni passé en `[Skip]` pour verdir la CI.
- ☐ Aucune donnée réelle, aucun secret, aucun domaine e-mail existant.
- ☐ Toute dépendance ajoutée vérifiée sur `nuget.org` / `npmjs.com`.
- ☐ Couverture **et** score de mutation publiés ensemble.

9 · SEUILS ET CHIFFRES À CONNAÎTRE

les seuls chiffres à citer en réunion

60 / 75 / 90 %	couverture Google : acceptable, louable, exemplaire
35-40 pts	écart attendu couverture ↔ score de mutation
~16 % / ~1,5 %	tests flaky / exécutions flaky chez Google
~84 %	des passages au rouge sont un flaky, pas une régression
~80 %	des échecs de disponibilité disparaissent au retry
3 jours / 3 mois	quarantaine max GitLab, puis suppression
5,2 % / 21,7 %	paquets hallucinés : commercial / open source
~40 % · 45 %	code IA vulnérable · tests Veracode avec faille
64 %	des erreurs des tests générés sont d'assertion
57,38 % · 16/50	ce qu'axe détecte : problèmes · critères WCAG
200-400 tokens	un snapshot d'accessibilité Playwright MCP
exit 2 · exit 9	seul code bloquant un hook · --minimum-expected-tests

p(95) < 800 sur `/api/products/search` · budget front < 170 Ko / < 5 s TTI. Au-delà de 8 blocages consécutifs, un hook `Stop` est outrepassé : aucun garde-fou logiciel n'est absolu.

10 · IA OU PAS IA ?

la grille de décision, tâche par tâche

Oracle disponible hors du code (spec, contrat, norme)	IA + validation contre la source
Aucun oracle documenté	Humain écrit l'oracle
Test rejoué des centaines de fois en CI	Script déterministe
Exploration one-shot d'une IHM inconnue	IA (agent exploratoire)
Faux négatif réglementaire ou financier	Les deux + mutation testing
Parsing et clustering de logs	Script (Drain, F-mesure 0,99)
Écrire des locators depuis le DOM réel	Les deux : MCP + IA
Décision de Go / No-Go	Humain — l'IA informe

11 · UN AGENT DANS LE PIPELINE

8 règles non négociables

- Secrets** — jamais en clair ; OIDC de préférence ; `show_full_output` désactivé.
- Permissions minimales** — `dontAsk`, `disallowedTools`, jeton à portée réduite.
- Pinning** — actions au **SHA complet**, version de modèle et de SDK épinglées.
- Timeout** — `timeout-minutes` sur chaque job ; le défaut est **6 heures**.
- Budget** — `--max-turns` explicite ; `total_cost_usd` relevé à chaque run.
- Artefacts** — l'agent consomme JUnit et traces ; il ne relance pas la suite.
- Idempotence** — `--bare` en CI : contexte explicite, jamais hérité du poste.
- PR de fork** — jamais `pull_request_target` + `checkout` du head, sans secret ni approbation.

L'agent est en **dernier étage** du pipeline et **ne bloque pas**. Le gating reste une règle écrite par la QA : seuils de couverture, seuils k6, violations axe critiques.

12 · RÉFÉRENCES

documentation officielle uniquement

Claude Code	code.claude.com/docs/en/
— bonnes pratiques	.../docs/en/best-practices
— hooks	.../docs/en/hooks
API — prompt caching	platform.claude.com/docs/en/build-with-claude/prompt-caching
Playwright — locators	playwright.dev/docs/locators
— agents · MCP	playwright.dev/docs/test-agents / mcp/introduction
.NET — dotnet test	learn.microsoft.com/dotnet/core/tools/dotnet-test-mtp
Angular · k6 seuils	angular.dev/guide/testing · grafana.com/docs/k6/latest/using-k6/thresholds/

DIAGNOSTIQUER UN ÉCHEC — UN SIGNAL DISCRIMINE CHAQUE BRANCHE

D'abord le dossier, jamais l'hypothèse : erreur brute non reformulée · --repeat-each exécuté · git log -1 + diff limités au SUT · contexte (workers, TZ, versions).

VRAI BUG PRODUIT

SIGNAL
Échoue partout, y compris hors CI.
Le produit dévie de la spec.

GESTE
Fiche d'anomalie.
Ne pas toucher au test.

TEST FAUX

SIGNAL
L'oracle vient du code.
--workers=1 ne change rien.

GESTE
Corriger le test.
Citer la spec dans l'assertion.

FLAKY

SIGNAL
n/N varie sur le même code.
--repeat-each=20 le prouve.

GESTE
Corriger la cause racine.
Jamais un retry, jamais un sleep.

ENVIRONNEMENT

SIGNAL
L'échec suit le runner, pas le test.
Il disparaît sur une autre machine.

GESTE
Corréler runner et échec
avant d'accuser l'infra.

Un test flaky est un test faux, pas une nuisance d'infrastructure — et quand il est le symptôme d'un défaut produit, le classement correct est « vrai bug ».

NE JAMAIS

- × Mettre une donnée réelle dans un prompt — même partielle, même « anonymisée ».
- × Committer un secret : `.env`, clé API, jeton OIDC restent hors du dépôt.
- × Mettre un test en `[Skip]` ou le supprimer pour faire passer la CI.
- × « Corriger » une flakiness par `waitForTimeout`, `Thread.Sleep` ou un `retry`.
- × Installer une URL, un paquet ou un serveur MCP suggéré sans vérification.
- × Prendre un LLM pour oracle : l'oracle est la spec, le contrat, la norme.
- × Laisser un self-healing sans journal ni revue humaine avant commit.
- × Croire une sortie d'outil sur parole — un outil renvoie « Done » même en échec.